

Double Linked List

Algoritma & Struktur Data

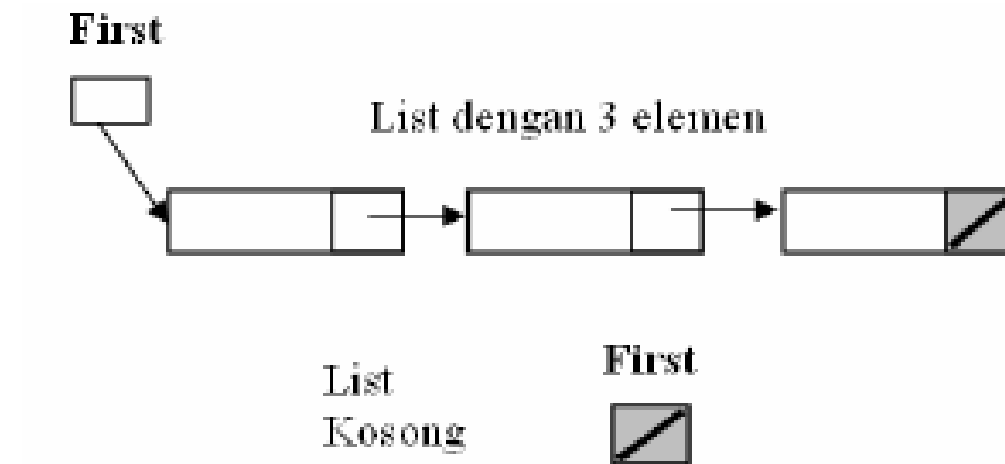
Outline Materi

- ▶ Doubly-linked List
- ▶ Doubly-linked List Tambah Diawal
- ▶ Doubly-linked List Tambah Diakhir
- ▶ Menghapus Doubly-linked List
- ▶ Circular Linked List



Single Linked List

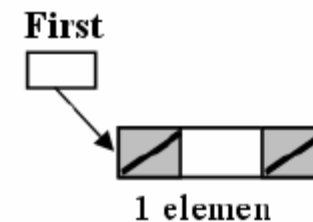
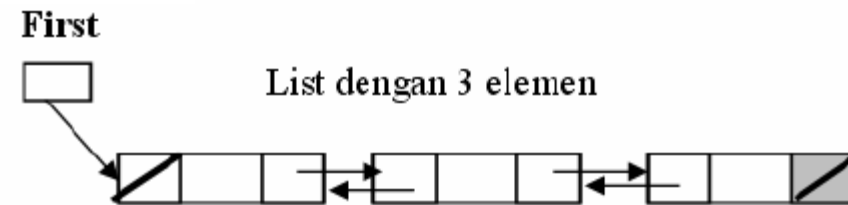
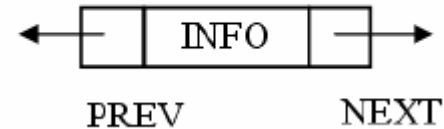
- ▶ Elemen pertama?
 - ▶ First = Node
- ▶ Elemen terakhir?
 - ▶ Node->Next = NULL
- ▶ List kosong?
 - ▶ First = NULL



Doubly-linked List

- ▶ Elemen pertama?
 - ▶ First = Node
 - ▶ Node->prev = NULL
- ▶ Elemen terakhir?
 - ▶ Node->next = NULL
- ▶ List Kosong?
 - ▶ First = NULL

Elemen List dengan pointer ganda :



Deklarasi Doubly Linked List

► Contoh deklarasi doubly linked list

```
struct tnode{  
    int data;  
    struct tnode *next, *prev;  
};
```



Penambahan Simpul di **Belakang**

A. Deklarasi tipe data simpul DLL

```
struct tnode {  
    int data;  
    struct tnode *prev, *next;  
} ;
```

B. Deklarasi pointer penunjuk DLL

```
int main() {  
    struct tnode *head, *tail, *node;  
    int bil;  
    ...  
    head = tail = NULL;  
    ...  
}
```



C. Membentuk simpul **baru** (pertama) dan mengaitkannya ke DLL [simpul tambah di belakang]

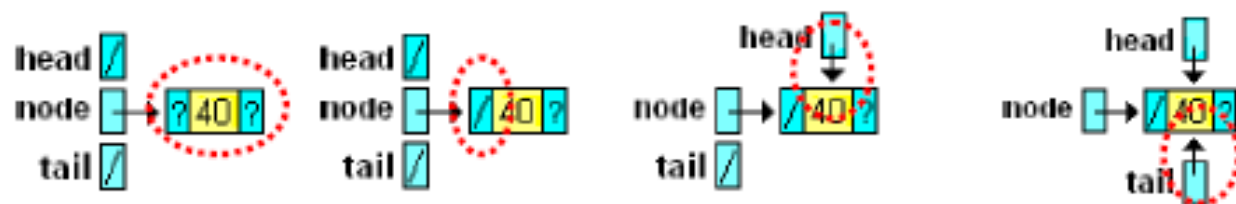
(c1) memesan sebuah simpul baru dengan menggunakan instruksi malloc() dan memasukkan data (nilai) ke simpul.

(c2) memberi nilai NULL kepada *field pointer* prev.

(c3) mengubah *pointer* head sehingga menunjuk ke simpul ini.

(c4) mengubah *pointer* tail sehingga menunjuk ke simpul ini.

```
scanf("%d", &bil);  
node = (struct tnode*) malloc (sizeof(struct tnode));  
node->data = bil;          /* c1 */  
if (head == NULL) {  
    node->prev = NULL;     /* c2 */  
    head = node;          /* c3 */  
    tail = node;          /* c4 */  
    tail->next = NULL;  
}
```



C. Membentuk simpul ke-n dan mengaitkannya ke DLL [simpul tambah di belakang]

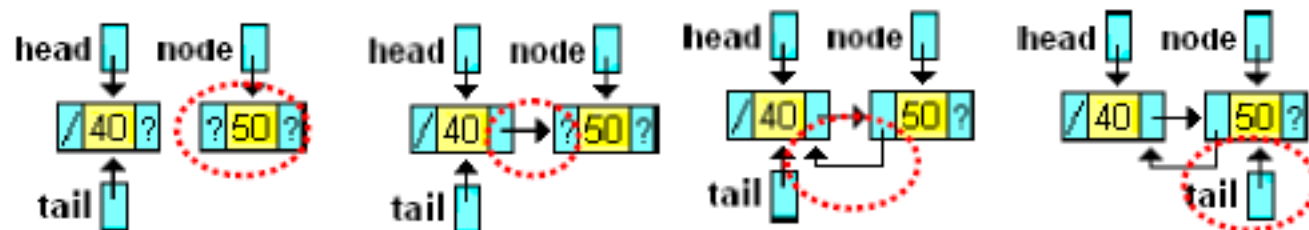
(c5) memesan sebuah simpul baru dengan menggunakan instruksi malloc(), memasukkan data (nilai) ke simpul, dan memberi nilai NULL kepada *field pointer* next

(c6) mengaitkan *field pointer* next pada simpul yang ditunjuk *pointer* tail ke simpul baru

(c7) mengaitkan *field pointer* prev pada simpul baru ke simpul yang ditunjuk *pointer* tail

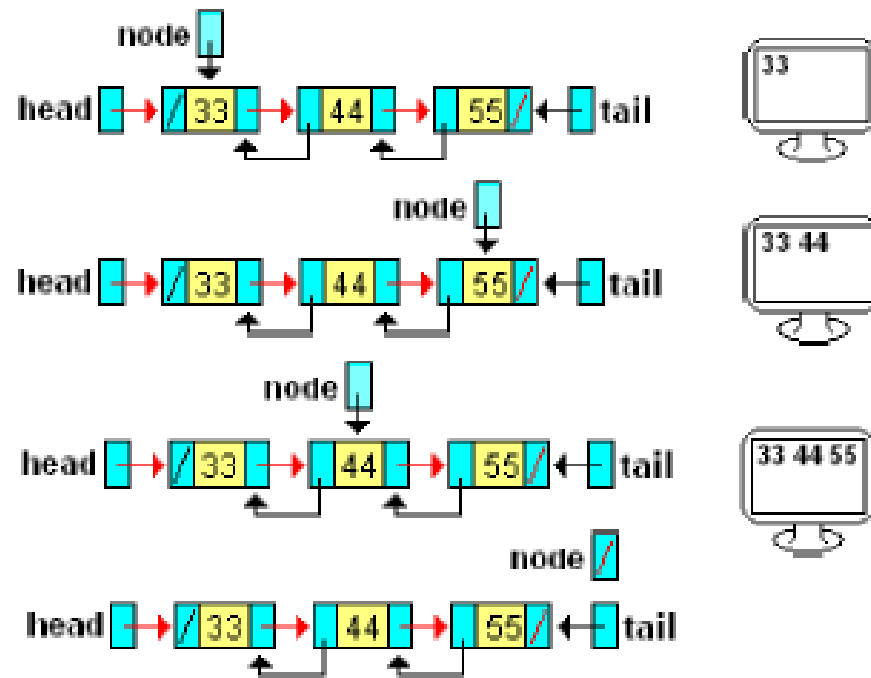
(c8) mengubah *pointer* tail sehingga menunjuk simpul baru

```
scanf("%d", &bil);  
node = (struct tnode*) malloc  
    (sizeof(struct tnode));  
node->data = bil;      /* c5 */  
if (head != NULL) {  
    tail->next = node;  /* c6 */  
    node->prev = tail;  /* c7 */  
    tail = node;       /* c8 */  
    tail->next = NULL;  
}
```



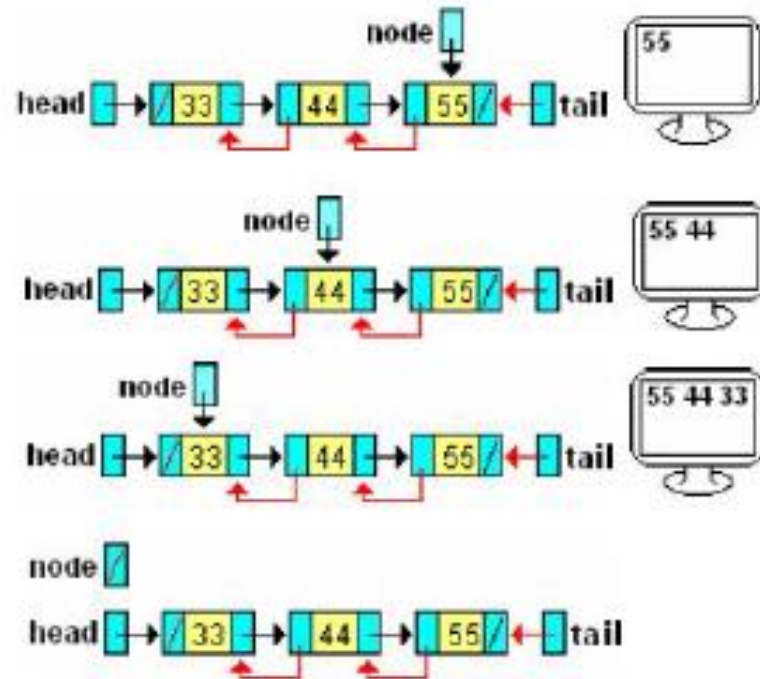
D1. MENCETAK ISI DLL (Head to Tail)

```
node = head;  
while (node != NULL) {  
    printf("%d ", node->data);  
    node = node->next;  
}
```



D2. MENCETAK ISI DLL (Tail to Head)

```
node = tail;  
while (node != NULL) {  
    printf("%d ", node->data);  
    node = node->prev;  
}
```



E1. MENGHAPUS DLL (Head to Tail)

```
while (head != NULL) {  
    node = head;  
    head = head->next;  
    free(node) ;  
}  
tail = NULL;
```

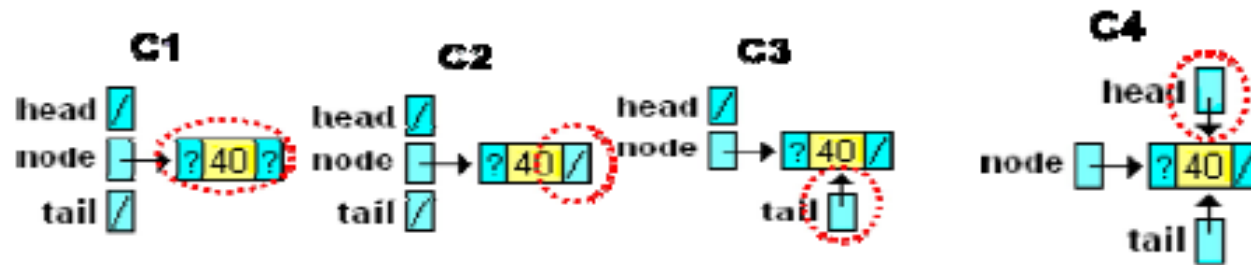
E2. MENGHAPUS DLL (Tail to Head)

```
while (tail != NULL) {  
    node = tail;  
    tail = tail->prev;  
    free(node) ;  
}  
head = NULL;
```



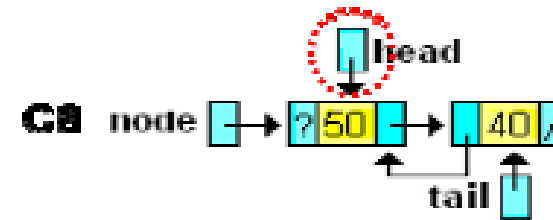
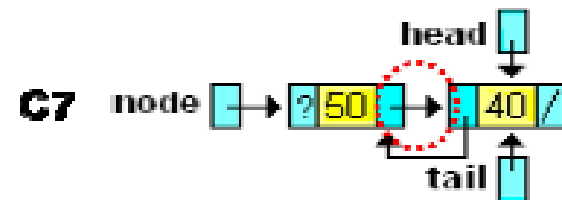
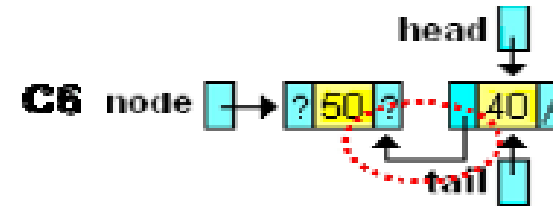
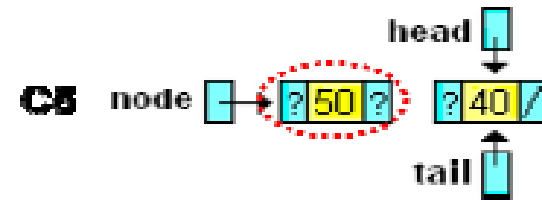
Penambahan Simpul di Depan

C. Membentuk simpul **baru** (pertama) dan mengaitkannya ke DLL [simpul tambah di depan]



```
scanf("%d", &bil);
node = (struct tnode*) malloc (sizeof(struct tnode));
node->data = bil;      /* c1 */
if (head == NULL) {
    node->next = NULL; /* c2 */
    tail = node;      /* c3 */
    head = node;      /* c4 */
    head->prev = NULL;
}
```

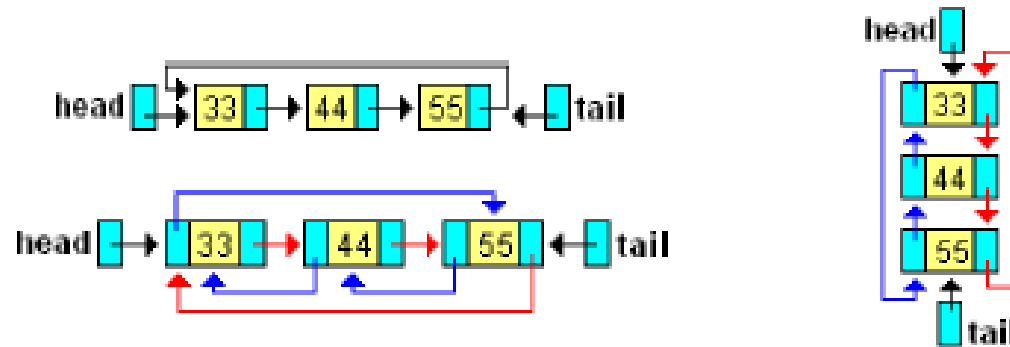
C. Membentuk simpul **ke-n** dan mengaitkannya ke DLL [simpul tambah di depan]



```
scanf("%d", &bil);
node = (struct tnode*) malloc (sizeof(struct tnode));
node->data = bil;      /* c5 */
if (head != NULL) {
    head->prev = node; /* c6 */
    node->next = head; /* c7 */
    head = node;       /* c8 */
    head->prev = NULL;
}
```

Circular Linked List

Circular linked list adalah linked list (SLL maupun DLL) yang simpul terakhirnya dikaitkan dengan simpul pertama



Latihan

- ▶ Diberikan double linked list sbb:

```
struct elebuku {  
    char kode[20];  
    char judul[50];  
    char penulis[20];  
    char penerbit[20];  
    int tahun;  
    struct elebuku *next, *prev;  
};
```

- ▶ Buatlah prosedur “tambahdepan” dan “tambahakhir” sesuai aturan “Double Linked List”
- ▶ Buatlah menu “Tampilkan isi Linked List” menjadi 2 menu berbeda, yaitu:
 - ▶ Tampilkan isi Linked List dari depan ke belakang
 - ▶ Tampilkan isi Linked List dari belakang ke depan



-
- ▶ Tambahkan menu “Ubah Data Buku”, dimana (Bonus jika menghapus node lama, dan menyisipkan node baru):
 - ▶ Input :
 - ▶ “Masukan kode buku : 12”
 - ▶ Output:
 - ▶ “Kode Buku : 12 “
 - ▶ “Judul :“
 - ▶ “Penulis :“
 - ▶ “Penerbit :“
 - ▶ “Tahun :“

